Pokročilé objektové technológie

prednáška

7



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

GUI – Graphical user interface
WPF

prof. Ing. **Emil Kršák**, PhD.

KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Katedra softvérových
technológií

Windows Presentation Foundation

WPF

Čo je WPF?

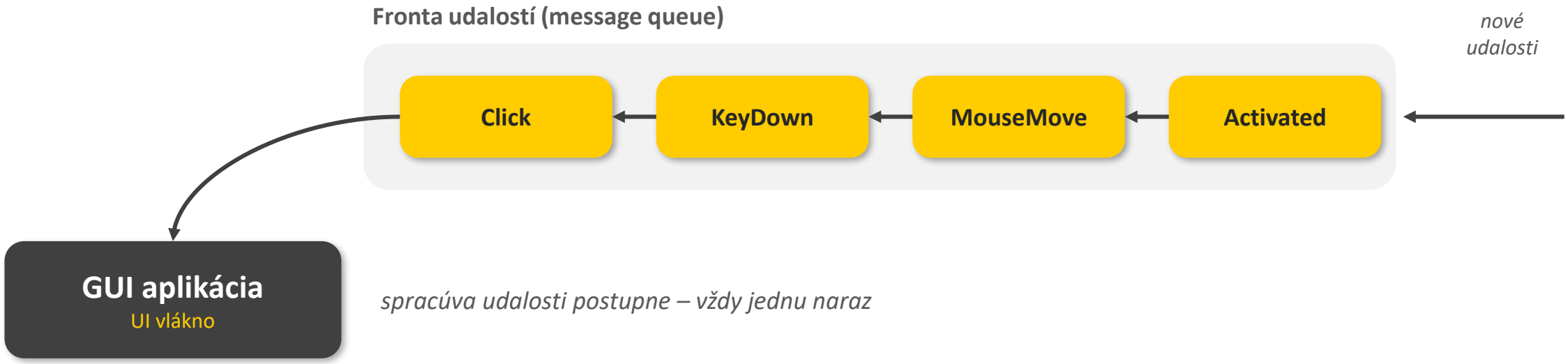
- Pôvodné („codename“) označenie „Avalon“, súčasť .NET 3.0 (2006)
- Hardvérovo akcelerované vykresľovanie (DirectX)
- Založený na vektorovej grafike (nezávislý od rozlíšenia)
- Tvorba GUI podobne ako WWW stránky
  - XAML – deklaratívny značkovací jazyk
- Oddelenie programátora od dizajnéra
  - Každý má iné vývojové prostredie (Visual Studio, Blend)
  - Podpora návrhových vzorov MVC, MVVM
- Prepracovaný data binding (predchodca vo WinForms)

WPF dnes

- Open source od roku 2018 (github.com/dotnet/wpf)
- Súčasť moderného .NET (od .NET Core 3.0, aktuálne .NET 10 LTS)
  - V .csproj: net10.0-windows a <UseWPF>true</UseWPF>
- Beží iba na Windows
- Stále široko používaný pre podnikové desktopové aplikácie
- Príbuzné technológie založené na XAML
  - WinUI 3 (Windows App SDK) – moderné natívne Windows GUI
  - .NET MAUI – multiplatformové (Windows, Android, iOS, macOS)
  - Avalonia UI – multiplatformová, syntax veľmi blízka WPF

Udalostne riadená aplikácia

- Myšlienka je veľmi jednoduchá
  - Akcie používateľa v GUI sú do programu prenášané udalosťami
  - Udalosti sú spracovávané postupne



  - Na tomto princípe funguje väčšina GUI

GUI udalosti

- Udalosti z myši
  - Pohyb myši (Mouse move)
  - Klik tlačidla myši (Mouse click)
  - Dvojitý klik myši (Mouse double-click)
- Klávesnica
  - Stlačenie klávesu (Key down)
  - Uvoľnenie stlačenia klávesu (Key up)
- Udalosti z okna
  - Stlačenie tlačidla v okne (Button click)
  - Výber položky menu (Menu selection)
  - Zmena aktívneho prvku v okne (Change in focus)
  - Okno do popredia – aktivácia (Window activation)
- ...

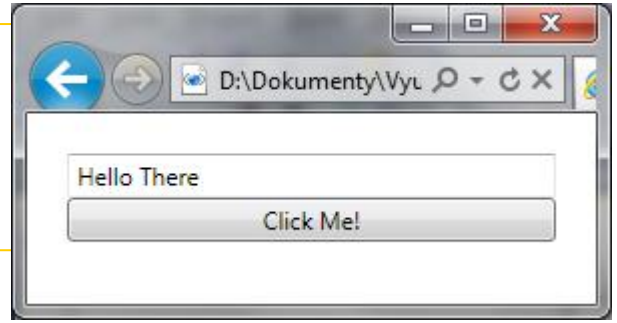
Extensible Application Markup Language

- XML elementy predstavujú prvky (Controls)
  - <Canvas /> objekt triedy Canvas
  - <Window /> objekt triedy Window
  - <Button /> objekt triedy Button
  - <TextBox /> objekt triedy TextBox



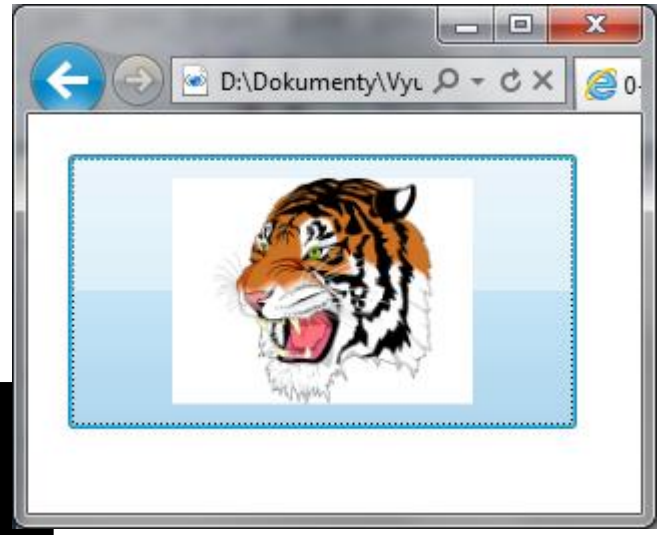

```
<Canvas xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
   <Button>Click Me!</Button>
</Canvas>
```

```
<StackPanel xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
   <TextBox>Hello There</TextBox>
   <Button>Click Me!</Button>
</StackPanel>
```



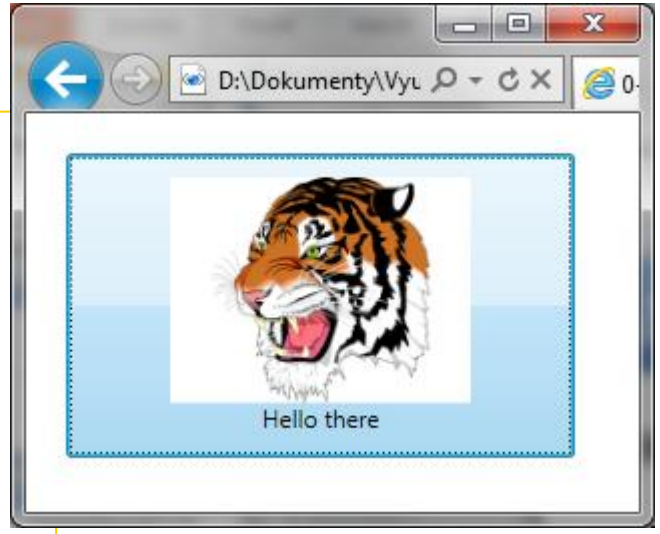
XAML – kompozícia

- Kompozitné prvky
  - Jednoduchý hierarchický model
  - Prvok môže obsahovať ďalšie prvky (jeden alebo viac, podľa typu prvku)




```
<Canvas xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
    <Button Margin="20" Padding="50 10">
        <Image Source="Images/tiger.png" Width="150" />
    </Button>
</Canvas>
```

```
<Canvas xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
   <Button Margin="20" Padding="50 10">
       <StackPanel>
           <Image Source="Images/tiger.png" Width="150" />
           <TextBlock TextAlignment="Center">Hello there</TextBlock>
        </StackPanel>
    </Button>
</Canvas>
```



XAML – kontajnerové prvky

- Kontajnerové prvky
  - Canvas: absolútne umiestňovanie (súradnice)
  - StackPanel: horizontálne alebo vertikálne ukladanie
  - DockPanel: dokovanie k hranám
  - Grid: umiestňovanie do riadkov a stĺpcov
  - UniformGrid: umiestňovanie do matice
  - WrapPanel: zalamovanie prvkov ako tok textu
- Používané ako hlavné (koreňové) prvky alebo ako vnorené prvky (ak chceme vložiť viac prvkov)

XAML – nastavovanie property

- Atribútová syntax (jednoduchšie a prehľadnejšie)

```
<TextBlock TextAlignment="Center" Text="Ahoj"/>
```

- Elementová syntax (pre zložité atribúty)

```
<TextBlock>
    <TextBlock.TextAlignment>
        Center
    </TextBlock.TextAlignment>
    <TextBlock.Text>
        Ahoj
    </TextBlock.Text>
</TextBlock>
```

Content atribút

- Content atribút (ak prvok obsahuje **ContentProperty**)
  - Element syntax

```
<Button>
    <Button.Content>
        Click me!
    </Button.Content>
</Button>
```



```
<Button>
    Click me!
</Button>
```

  - Attribute syntax

```
<Button Content="Click me!">
```

Content atribút – kolekcia

- Implicitná kolekcia (ak je Content typu kolekcia)

```
<ListBox>
    <ListBox.Items> <!-- ListBox.Items môžeme vynechať -->
        <ListBoxItem Content="aaa" />
        <ListBoxItem Content="bbb" />
    </ListBox.Items>
</ListBox>
```

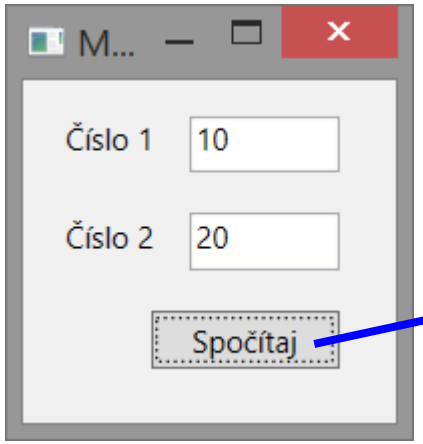
- Bez atribútu kolekcie

```
<ListBox>
    <ListBoxItem Content="aaa" />
    <ListBoxItem Content="bbb" />
</ListBox>
```

Code-behind

- Pre každú udalosť môžeme definovať obslužnú metódu (event handler) – kód na pozadí (code behind)
- Dizajnér – navrhuje okná
- Programátor – tvorí kód na pozadí

```
<Window x:Class="POT2016_WPF_code_behind.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml" Title="MainWindow" SizeToContent="WidthAndHeight">
  <Grid Margin="10">
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="Auto"/>
      <ColumnDefinition Width="60"/>
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
      <RowDefinition/>
      <RowDefinition/>
      <RowDefinition/>
    </Grid.RowDefinitions>
    <Label Content="Číslo 1"/>
    <TextBox x:Name="txtCislo1" Grid.Column="1" Margin="4" Text="0"/>
    <Label Content="Číslo 2" Grid.Row="1"/>
    <TextBox x:Name="txtCislo2" Grid.Row="1" Grid.Column="1" Margin="4" Text="0"/>
    <Button Content="Spočítaj" Grid.Row="2" Grid.ColumnSpan="2" Margin="4" Click="BtnSpocitaj_Click"/>
  </Grid>
</Window>
```



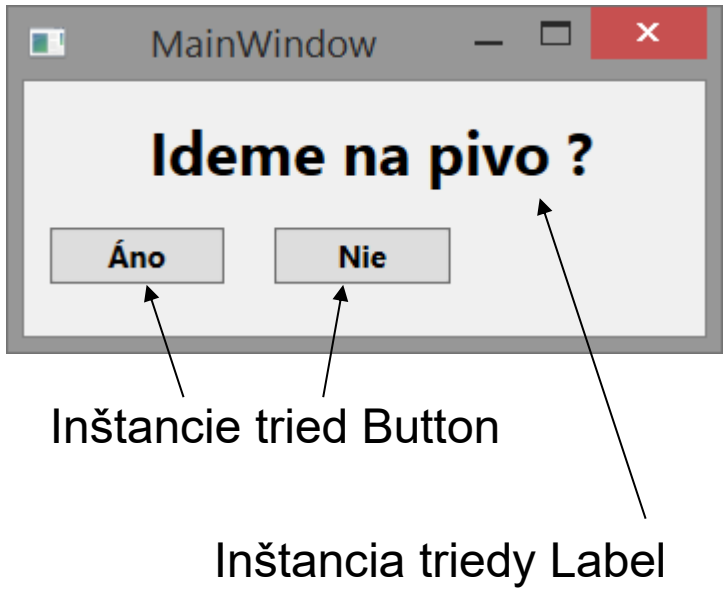

```
public partial class MainWindow : Window
{
  public MainWindow()
  {
    InitializeComponent();
  }

  private void BtnSpocitaj_Click(object sender, RoutedEventArgs e)
  {
    if (int.TryParse(txtCislo1.Text, out int a) &&
        int.TryParse(txtCislo2.Text, out int b))
      MessageBox.Show($"Súčet = {a + b}");
    else
      MessageBox.Show("Zadajte celé čísla.");
  }
}
```

Code-behind – dynamika

- Okamžitá zmena vzhľadu programovo

```
<Window x:Class="POT_WPF_code_behind2.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Background="{x:Static SystemColors.ControlBrush}" Title="MainWindow" Height="139" Width="286">
    <Canvas>
        <Label x:Name="lblOtazka" Canvas.Left="45" Canvas.Top="7" Content="Ideme na pivo?" FontWeight="Bold" FontSize="24" />
        <Button x:Name="btnAno" Canvas.Left="10" Canvas.Top="58" Content="Áno" Width="70" FontWeight="Bold" />
        <Button x:Name="btnNie" Canvas.Left="100" Canvas.Top="58" Content="Nie" Width="70" FontWeight="Bold" />
    </Canvas>
</Window>
```




```
public partial class MainWindow : Window
{
  public MainWindow()
  {
    InitializeComponent();
    btnAno.MouseEnter += BtnAno_MouseEnter;
    btnNie.Click += BtnNie_Click;
  }

  private void BtnAno_MouseEnter(object sender, MouseEventArgs e)
  {
    double x = Canvas.GetLeft(btnAno);
    Canvas.SetLeft(btnAno, x < 50 ? 190 : 10);
  }

  private void BtnNie_Click(object sender, RoutedEventArgs e)
  {
    MessageBox.Show("Tak seď a dávaj pozor!");
  }
}
```

Viacvláknové aplikácie

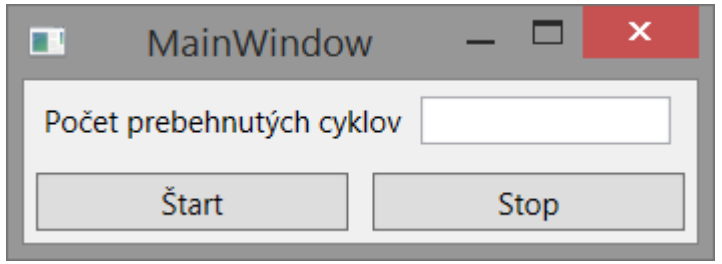
- GUI beží v jednom vlákne (UI thread)
- GUI nedovolí modifikovať elementy z iných vlákien (končí výnimkou)

```
public partial class MainWindow : Window
{
  public MainWindow() {
    InitializeComponent();
  }
  private CancellationTokenSource? cts;

  private void button1_Click(object sender, RoutedEventArgs e)
  {
    if (cts == null) {
      cts = new CancellationTokenSource();
      var token = cts.Token;
      Task.Run(() => Pocitanie(token));
    }
  }

  private void button2_Click(object sender, RoutedEventArgs e)
  {
    cts?.Cancel();
    cts = null;
  }

  private void Pocitanie(CancellationToken token)
  {
    for (long i = 0; !token.IsCancellationRequested; i++) {
      double d = Math.Sin(i);
      if (i % 10 == 0) textBox1.Text = i.ToString(); // výnimka!
    }
  }
}
```




```
private void Pocitanie()
{
  for (long i = 0; i < long.MaxValue; i++)
  {
    double d = Math.Sin(i);
    if (i % 10 == 0) textBox1.Text = d.ToString();
  }
}
```

InvalidOperationException was unhandled

The calling thread cannot access this object because a different thread owns it.

Troubleshooting tips:

Get general help for this exception.

Search for more Help Online...

Actions:

View Detail...

Copy exception detail to the clipboard

Viacvláknové aplikácie – preposlanie

- Riešenie – preposlať aktualizáciu GUI do UI vlákna
- Využiť Dispatcher (fronta úloh UI vlákna)

```
private void Pocitanie(CancellationToken token)
{
  for (long i = 0; !token.IsCancellationRequested; i++)
  {
    double d = Math.Sin(i);
    if (i % 10 == 0)
      Dispatcher.Invoke(() => textBox1.Text = i.ToString());
  }
}
```

Metódy Dispatchera

`Invoke(...)`

synchrónne – čaká, kým UI vlákno akciu vykoná

`InvokeAsync(...)`

asynchrónne – vráti sa hneď

`CheckAccess()`

true, ak sme v UI vlákne

Ešte lepšie: async/await (ďalší slajd)

Viacvláknové aplikácie – async/await

```
private CancellationTokenSource? cts;

private async void button1_Click(object sender, RoutedEventArgs e)
{
  if (cts != null) return;
  cts = new CancellationTokenSource();
  var token = cts.Token;
  var progress = new Progress<long>(i => textBox1.Text = i.ToString());
  //                                └──────────── callback ────────────┘
  try {
    await Task.Run(() => Pocitanie(progress, token), token);
  }
  catch (OperationCanceledException) {
    textBox1.Text += " (zastavené)";
  }
  finally {
    cts.Dispose();
    cts = null;
  }
}

private void button2_Click(object sender, RoutedEventArgs e) => cts?.Cancel();

private static void Pocitanie(IProgress<long> progress, CancellationToken token)
{
  for (long i = 0; ; i++) {
    token.ThrowIfCancellationRequested();
    double d = Math.Sin(i);
    if (i % 100_000 == 0) progress.Report(i);
  }
}
```

await

UI vlákno nie je blokované; kód za await pokračuje opäť v UI vlákne

Progress<T>

callback sa vykoná vo vlákne, kde bol objekt vytvorený (tu UI vlákno) - SynchronizationContext

CancellationToken

kooperatívne zrušenie – náhrada za Thread.Abort(), alebo "volatile bool"

async void

iba pre obsluhu udalostí, inak async Task

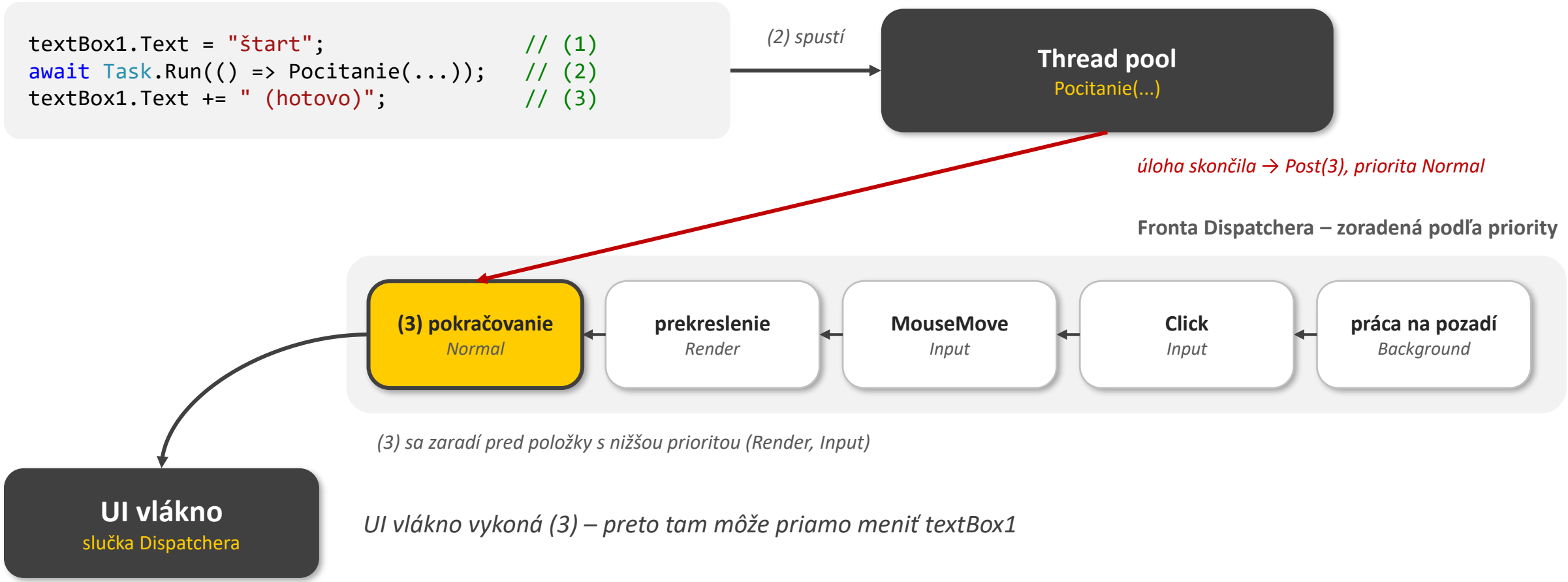
Viacvláknové aplikácie – porovnanie

	Slajd 15 `Task.Run`	Slajd 16 `Dispatcher`	Slajd 17 `async/await`
Spustenie	`Task.Run`	`Task.Run`	`await Task.Run`
Aktualizácia GUI	✗ priamo – výnimka	`Dispatcher.Invoke`	✓ `Progress<T>`
Výpočet pozná GUI?	✗ áno	✗ áno	✓ nie
Vieme, kedy úloha skončila?	✗ nie – `Task` sa zahodí	✗ nie – `Task` sa zahodí	✓ áno – kód za `await`
Spracovanie chýb	✗ stratia sa	✗ stratia sa	✓ `try/catch` ako v synchrónnom kóde

`async/await` – asynchrónny kód, ktorý sa číta ako obyčajný sekvenčný kód

Čo presne robí await

```
textBox1.Text = "štart";                  // (1)
await Task.Run(() => Pocitanie(...));     // (2)
textBox1.Text += " (hotovo)";             // (3)
```



`await` neblokuje – metóda sa rozdelí na dve položky vo fronte, medzi nimi UI vlákno spracúva kliky aj prekresľovanie



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Katedra softvérových
technológií

WPF DATABINDING

WPF Data Binding

```
<TextBox Text="{Binding City,
    UpdateSourceTrigger=PropertyChanged}" />
```



binding

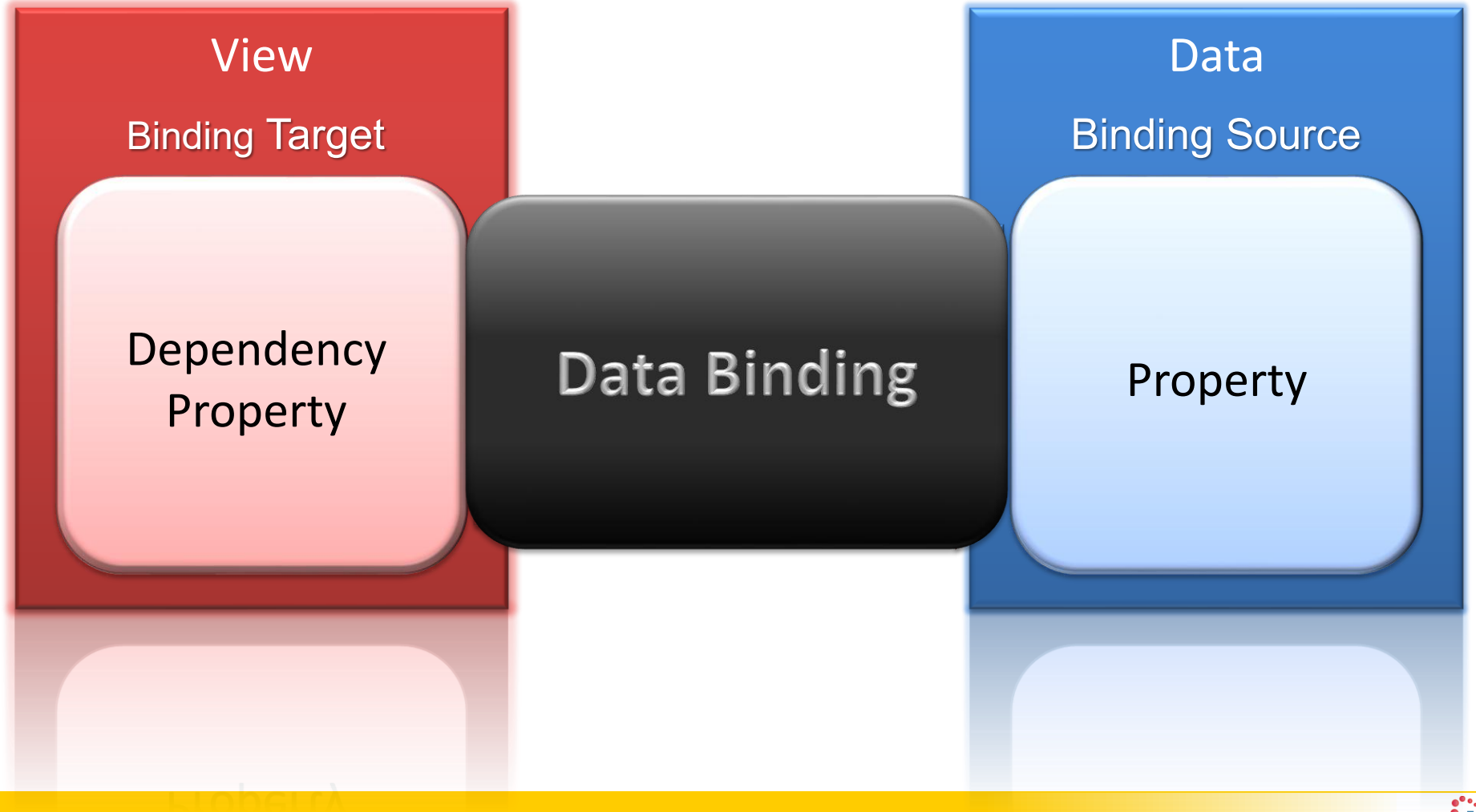
```
class MainWindowViewModel : INotifyPropertyChanged
{
  private string city = "";
  public string City
  {
    get => city;
    set { city = value; OnPropertyChanged(); }
  }

  public event PropertyChangedEventHandler? PropertyChanged;

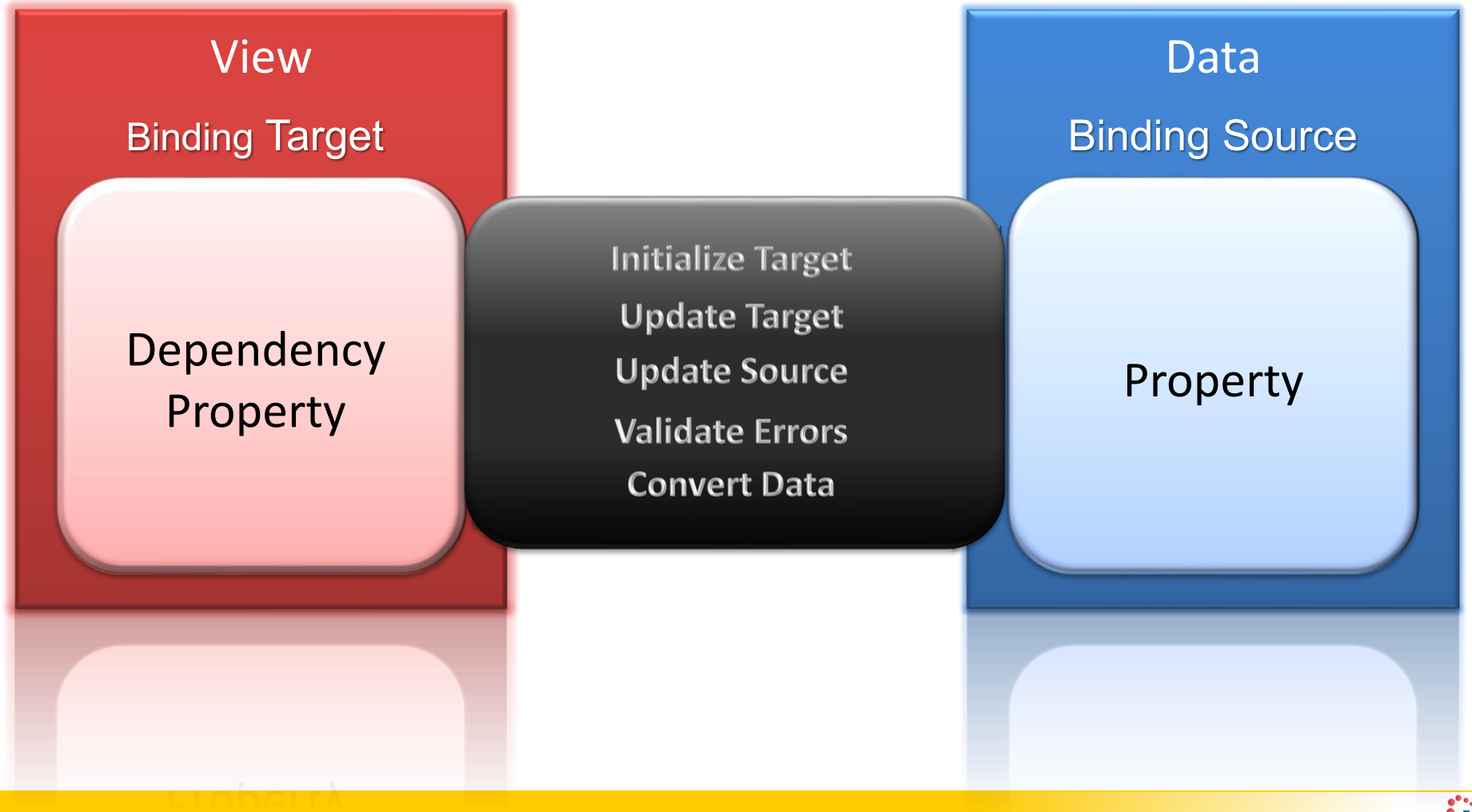
  void OnPropertyChanged([CallerMemberName] string? n = null)
    => PropertyChanged?.Invoke(this, new(n));
}

// v konštruktore okna (code-behind):
DataContext = new MainWindowViewModel();
```

Data binding – princíp



Data binding – princíp



Data binding

- Prenos dát medzi GUI a aplikačnou logikou
  - Target: takmer ľubovoľná property ľubovoľného elementu WPF
  - Source: public property objektu:
    - takmer ľubovoľná property ľubovoľného elementu WPF
    - CLR – vlastné objekty a ich property
    - XAML property
    - ADO DataColumns (TypeDescriptor)
    - XML Node
  - Dynamika – aktualizácia GUI pri zmene property
    - DependencyProperty – všetky property objektov z XAML
    - INotifyPropertyChanged – CLR (vlastné objekty)
    - INotifyCollectionChanged – pre zmeny počtu prvkov v kolekcii
  - Value Converter (IValueConverter) – konverzia hodnôt

Data binding

- Jednoduchý zápis binding v XAML

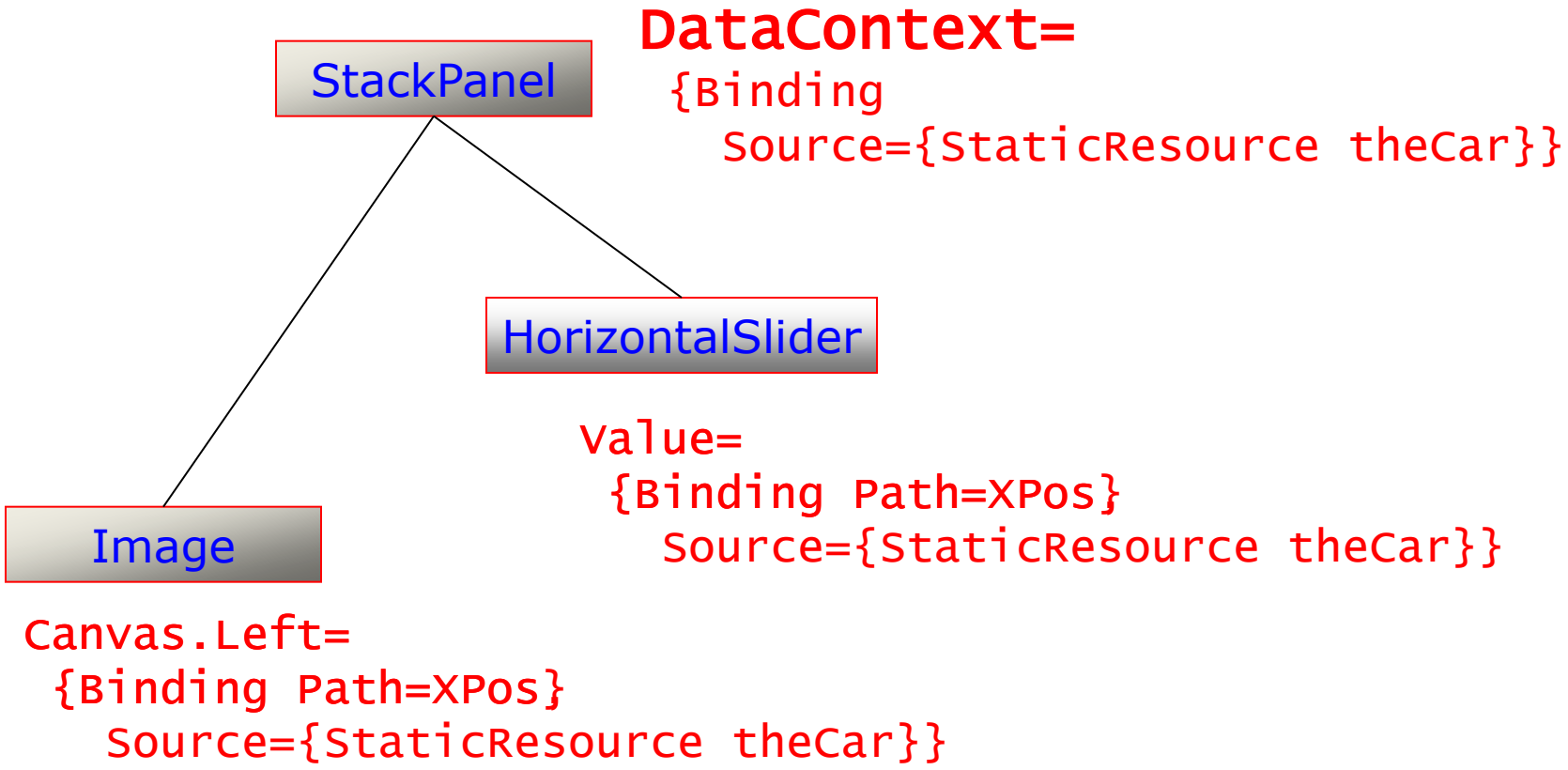
```
<Canvas xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
 <Slider Name="slider" />
 <Label Canvas.Top="50">
  <Label.Content>
   <Binding Path="Value" ElementName="slider" />
  </Label.Content>
 </Label>
</Canvas>
```

```
<Canvas xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml" >
 <Slider Name="slider" />
 <Label Canvas.Top="50" Content="{Binding Path=Value, ElementName=slider}" />
</Canvas>
```

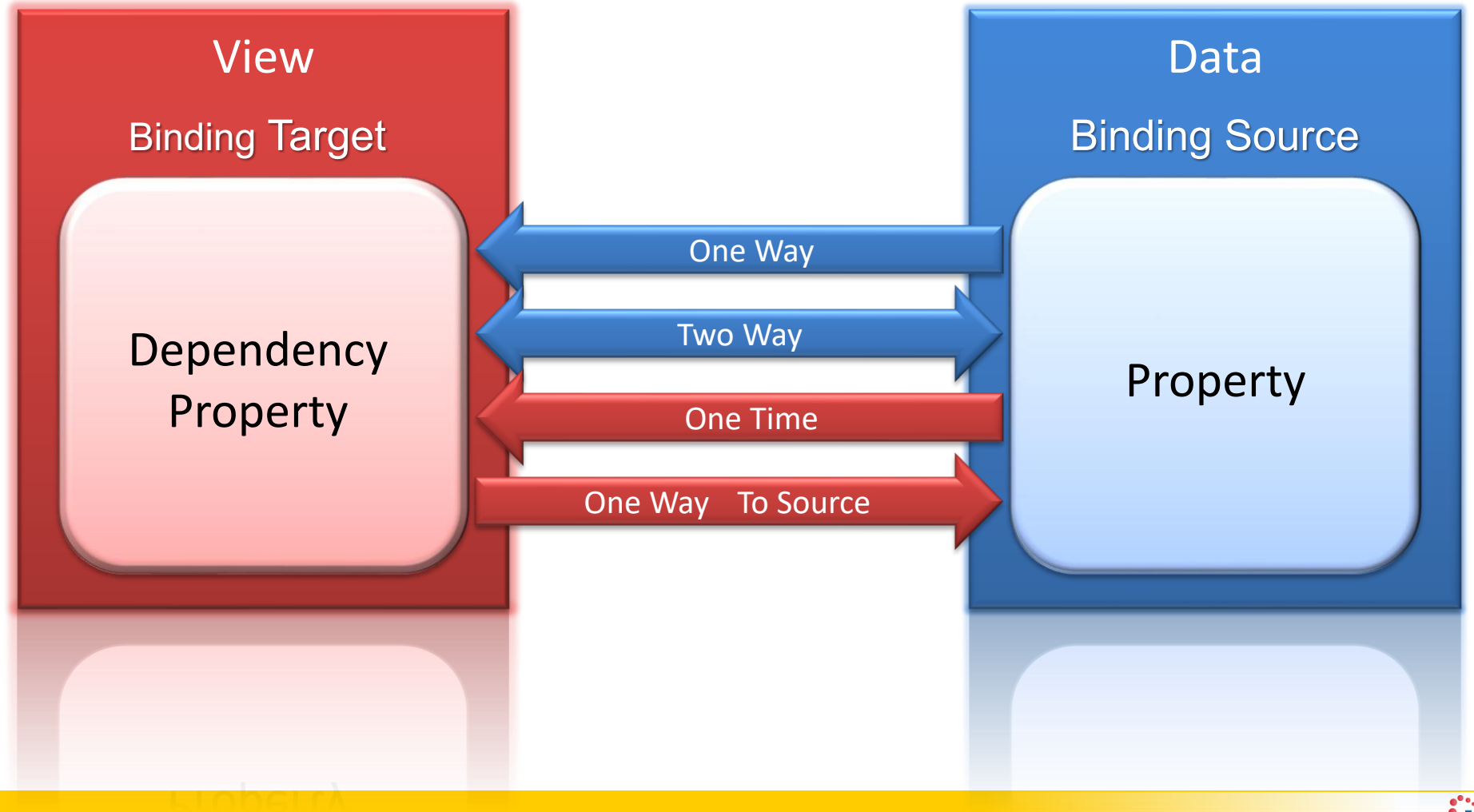
- Ukážky (binding2a, binding2b)

Data binding – DataContext

- DataContext – všeobecný DataSource
- Je prenášaný na prvky vnorené



Data binding – princíp



Data binding – typ väzby

- Možné typy
  - OneWay
  - TwoWay
  - OneTime
  - OneWayToSource
- OneWayToSource a OneTime – source objekt môže byť ľubovoľný objekt – jednoduché čítanie/zápis property
- OneWay a TwoWay využívajú rozhranie INotifyPropertyChanged
- Kolekcie (napr. ListBox) využívajú INotifyCollectionChanged

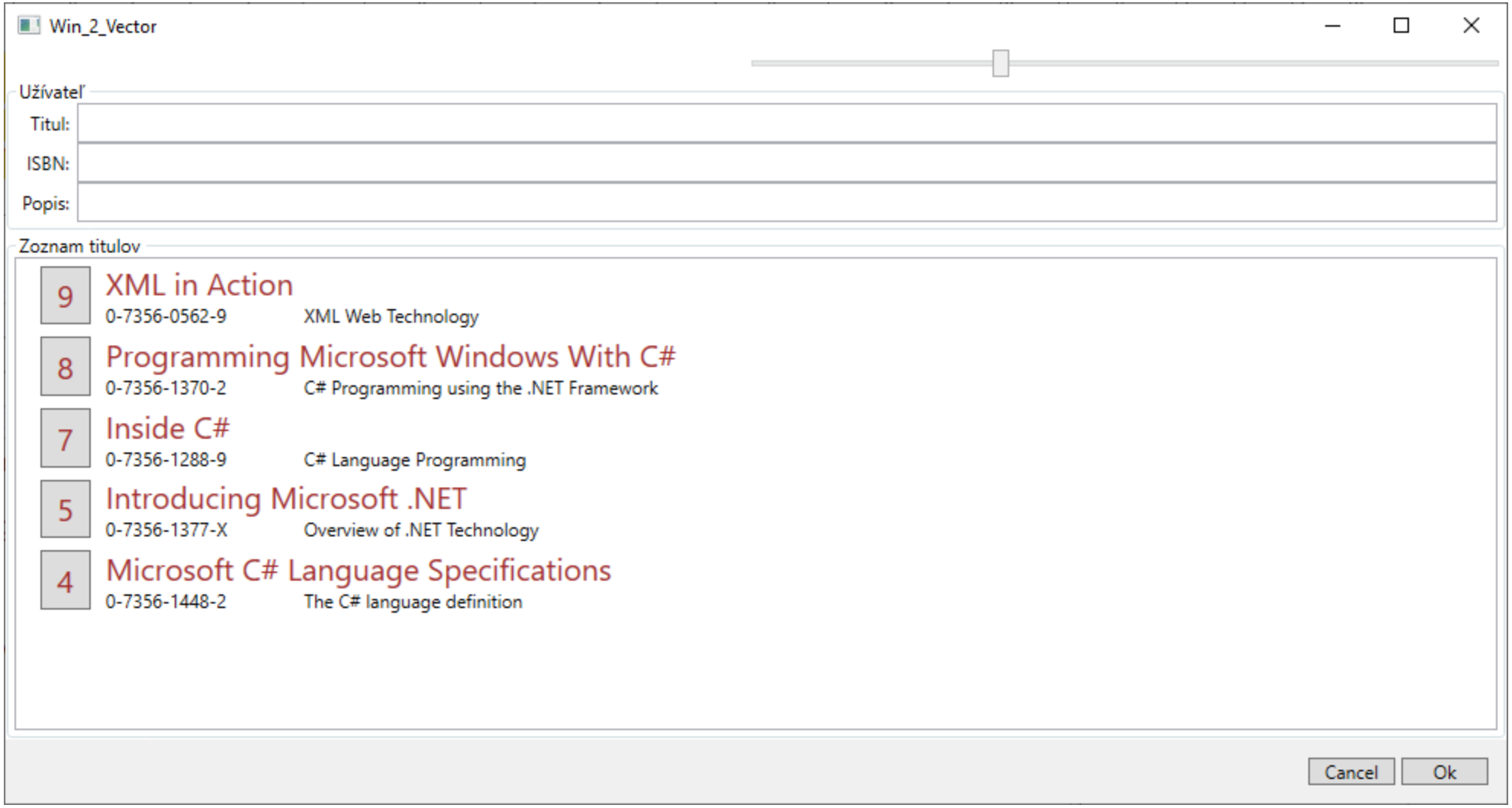
Štýly

- Podobné ako CSS
- Možnosť zmeniť vzhľad celého GUI (themes)

```
<Canvas xmlns="..." xmlns:x="...">
  <TextBox>
    <TextBox.Style>
      <Style TargetType="TextBox"><Setter Property="FontSize" Value="18"/></Style>
    </TextBox.Style>
  </TextBox>
</Canvas>
```

```
<Canvas xmlns="..." xmlns:x="...">
  <Canvas.Resources>
    <Style x:Key="MyTextBox" TargetType="TextBox">
      <Setter Property="FontSize" Value="18" />
    </Style>
    <!-- implicitný štýl (bez x:Key) – platí pre všetky Button -->
    <Style TargetType="Button">
      <Setter Property="Margin" Value="5" />
    </Style>
  </Canvas.Resources>
  <TextBox Style="{StaticResource MyTextBox}" />
  <Button Content="OK" />
</Canvas>
```

Aplikácia bez code-behind



Dáta vo ViewModeli

```
public class Book
{
    public string Isbn { get; init; } = "";
    public string Title { get; init; } = "";
    public string Summary { get; init; } = "";
    public int Number { get; init; }
}

public class LibraryViewModel
{
    public ObservableCollection<Book> Books { get; } =
    [
        new() { Isbn = "0-7356-0562-9", Number = 9, Title = "XML in Action",
                Summary = "XML Web Technology" },
        new() { Isbn = "0-7356-1370-2", Number = 8,
                Title = "Programming Microsoft Windows With C#",
                Summary = "C# Programming using the .NET Framework" },
    ];
}

<!-- MainWindow.xaml: DataContext v XAML, code-behind ostáva prázdny -->
<Window.DataContext>
    <local:LibraryViewModel />
</Window.DataContext>
```

Template

```
<ListBox x:Name="Zoznam" ItemsSource="{Binding Books}">
 <ListBox.ItemTemplate>
  <DataTemplate>
   <StackPanel Orientation="Horizontal">
    <Button Content="{Binding Number}"
      Padding="10 5" Margin="10 2" FontSize="20" Foreground="Brown"/>
    <StackPanel>
     <TextBlock Text="{Binding Title}"
         FontSize="20" Foreground="Brown"/>
     <StackPanel Orientation="Horizontal">
      <TextBlock Text="{Binding Isbn}" Width="130"/>
      <TextBlock Text="{Binding Summary}"/>
     </StackPanel>
    </StackPanel>
   </StackPanel>
  </DataTemplate>
 </ListBox.ItemTemplate>
</ListBox>
```

Template položky

Animácie

- Na animáciu sa používa Storyboard
  - Definuje timeline

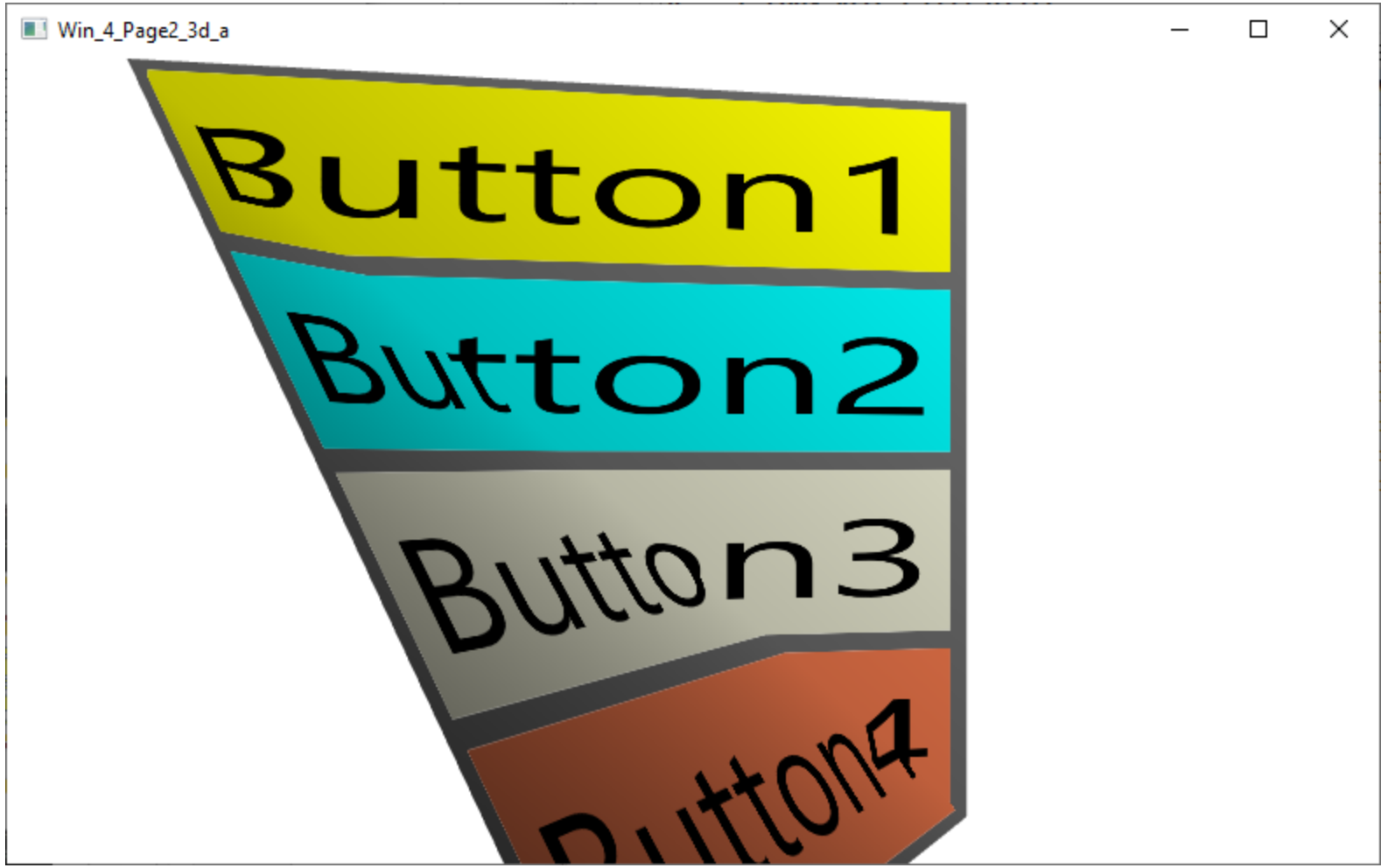
```
<Page xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
 <Rectangle Name="MyRectangle" Width="100" Height="100" Fill="Blue">
  <Rectangle.Triggers>
   <EventTrigger RoutedEvent="Rectangle.Loaded">
    <BeginStoryboard>
     <Storyboard>
      <DoubleAnimation
       Storyboard.TargetName="MyRectangle"
       Storyboard.TargetProperty="Width" From="100" To="200" Duration="0:0:5"
       AutoReverse="True" RepeatBehavior="Forever" />
     </Storyboard>
    </BeginStoryboard>
   </EventTrigger>
  </Rectangle.Triggers>
 </Rectangle>
</Page>
```

Animácie – komplexnejšie

- Viac animácií v jednom Storyboarde pre ten istý prvok
- Bežia paralelne

```
<Page xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
 <Rectangle Name="MyRectangle" Width="100" Height="100" Fill="Blue">
  <Rectangle.Triggers>
   <EventTrigger RoutedEvent="Rectangle.Loaded">
    <BeginStoryboard>
     <Storyboard>
      <DoubleAnimation
       Storyboard.TargetName="MyRectangle" Storyboard.TargetProperty="Width"
       From="100" To="200" Duration="0:0:2" AutoReverse="True" RepeatBehavior="Forever" />
      <DoubleAnimation
       Storyboard.TargetName="MyRectangle"
       Storyboard.TargetProperty="Height"
       From="100" To="50" Duration="0:0:3"
       AutoReverse="True" RepeatBehavior="Forever" />
     </Storyboard></BeginStoryboard></EventTrigger></Rectangle.Triggers></Rectangle></Page>
```

3D ukážka



3D modely

```
<Page   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      Height="400" Width="500">
  <Viewport3D Name="mainViewport" ClipToBounds="True">
    <Viewport3D.Camera>
      <PerspectiveCamera Position="0,0,4" />
    </Viewport3D.Camera>
    <Viewport2DVisual3D >
      <Viewport2DVisual3D.Transform>
        <RotateTransform3D>
          <RotateTransform3D.Rotation>
            <AxisAngleRotation3D x:Name="uiRotate" Angle="40" Axis="0, 1, 0" />
          </RotateTransform3D.Rotation>
        </RotateTransform3D>
      </Viewport2DVisual3D.Transform>
      <Viewport2DVisual3D.Geometry>
        <MeshGeometry3D Positions="-1,1,0 -1,-1,0 1,-1,0 1,1,0" TextureCoordinates="0,0 0,1 1,1 1,0" TriangleIndices="0 1 2 0 2 3"/>
      </Viewport2DVisual3D.Geometry>
      <Viewport2DVisual3D.Material>
        <DiffuseMaterial Viewport2DVisual3D.IsVisualHostMaterial="True" Brush="White"/>
      </Viewport2DVisual3D.Material>
      <Viewport2DVisual3D.Visual>
        <StackPanel Orientation="Vertical">
          <Button Background="Yellow" >Button1</Button>
          <Button Background="Aqua" >Button2</Button>
          <Button Background="Beige" >Button3</Button>
          <Button Background="Coral" >Button4</Button>
        </StackPanel>
      </Viewport2DVisual3D.Visual>
    </Viewport2DVisual3D>
    <ModelVisual3D>
      <ModelVisual3D.Content>
        <DirectionalLight Color="#FFFFFFFF" Direction="0,0,-1"/>
      </ModelVisual3D.Content>
    </ModelVisual3D>
  </Viewport3D>
</Page>
```

Kamera

Transformácia - otočenie

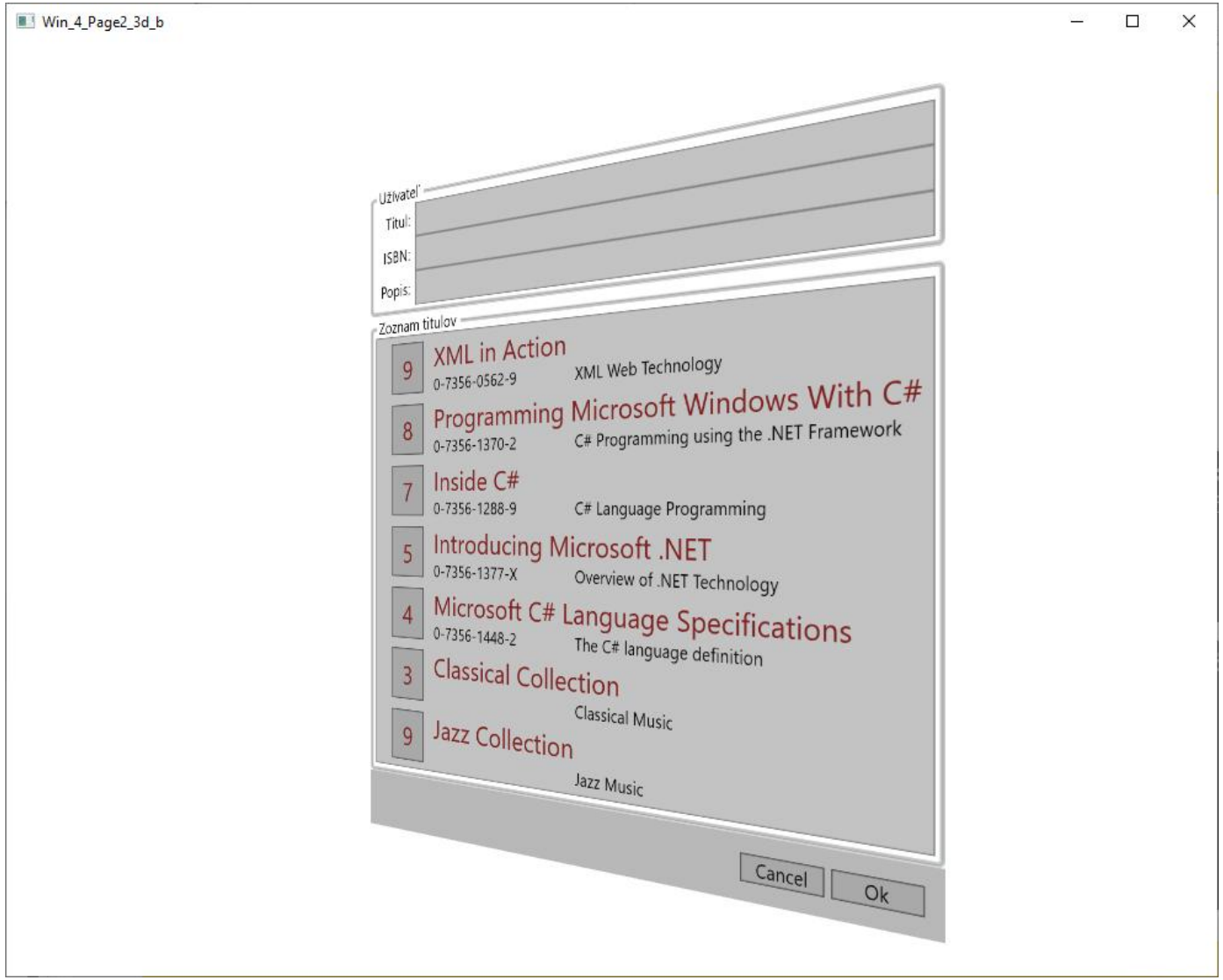
Kostra

Textúra

Vizuál

Svetlo

3D ukážka 2

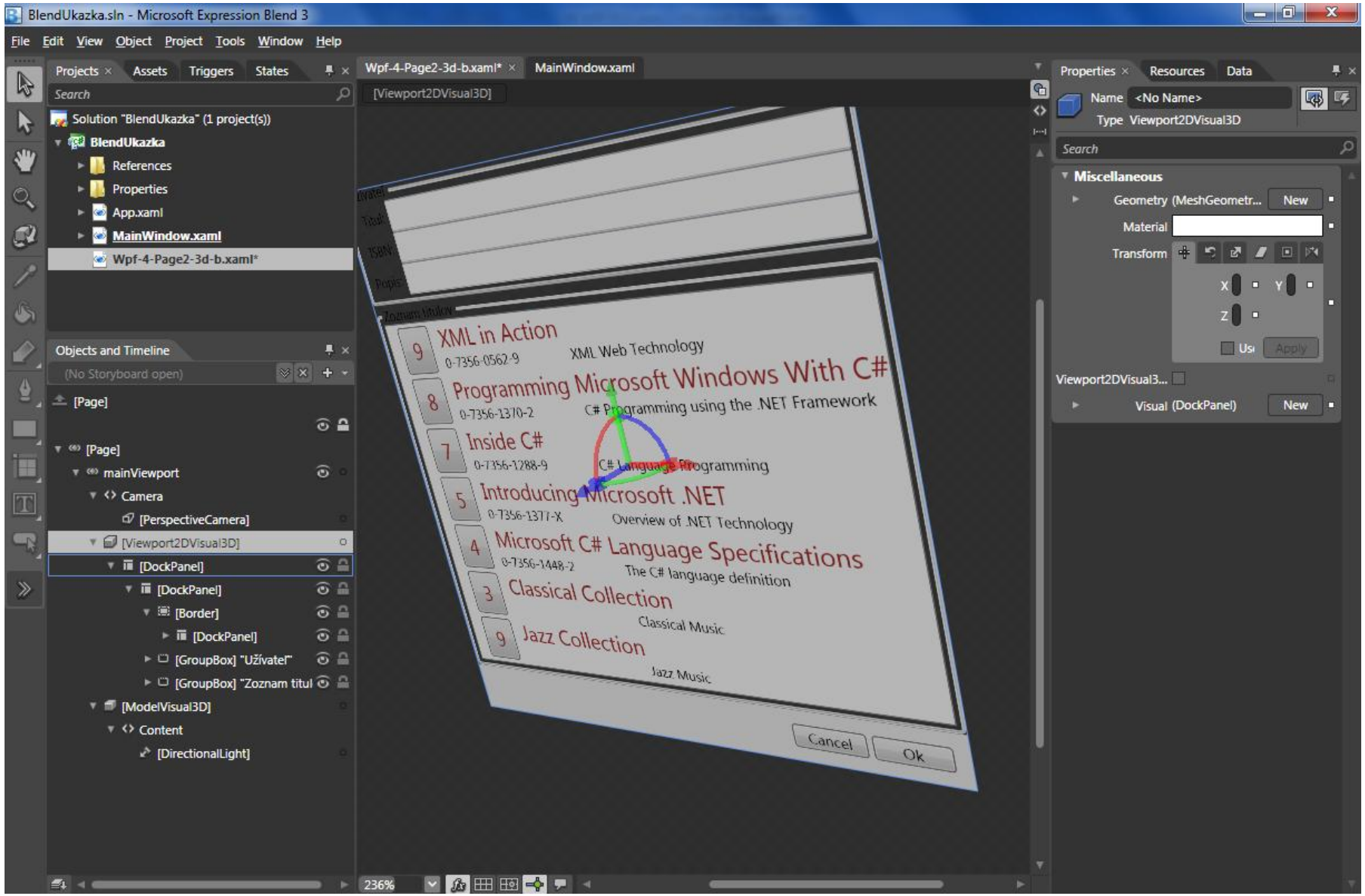

Win_4_Page2_3d_b
Užívateľ
Titul:
ISBN:
Popis:
Zoznam titulov
9
XML in Action
0-7356-0562-9
XML Web Technology
8
Programming Microsoft Windows With C#
0-7356-1370-2
C# Programming using the .NET Framework
7
Inside C#
0-7356-1288-9
C# Language Programming
5
Introducing Microsoft .NET
0-7356-1377-X
Overview of .NET Technology
4
Microsoft C# Language Specifications
0-7356-1448-2
The C# language definition
3
Classical Collection
Classical Music
9
Jazz Collection
Jazz Music
Cancel
Ok

Multimédia

- Práca s audio/video
  - <MediaElement />
  - Je vlastne wrapper nad Windows Media Player

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Width="800" WindowState="Maximized">
 <StackPanel>
  <StackPanel Orientation="Horizontal">
   <MediaElement Source="Media/intro.mp4" Width="500" />
   <MediaElement Source="Media/xbox.mp4" Width="500" />
  </StackPanel>
  <StackPanel Orientation="Horizontal">
   <MediaElement Source="Media/numbers.mp4" Width="500" />
   <MediaElement Source="Media/bee.mp4" Width="500" />
  </StackPanel>
 </StackPanel>
</Window>
```

Blend for Visual Studio (pôv. Expression Blend)



Ako tvoriť desktop aplikácie s GUI

- Aplikačná logika priamo v GUI
  - Rýchly vývoj
  - Výmena dát: metódy, udalosti (events)
  - Vhodné pre malé projekty
- Aplikačná logika oddelená od GUI
  - Vyššia počiatočná réžia
  - Výmena dát: binding (ak GUI umožňuje), udalosti
  - Pre veľké projekty prehľadnejšie
  - Podpora unit testov
  - Znovupoužiteľnosť aplikačnej logiky (klient, server, ...)



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Katedra softvérových
technológií

emil.krsak@uniza.sk

ĎAKUJEM ZA POZORNOSŤ

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov Fakulty riadenia a informatiky Žilinskej univerzity v Žiline

- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

prof. Ing. **Emil Kršák**, PhD.

emil.krsak@uniza.sk